

Lab6实验报告

思考题

Thinking 6.1

父进程先关闭写通道

```
father_process
    close(filides[1]); /* write end is unused */
    read(filides[0], buf, 100); /* Get data from pipe */
    printf("father-process read:%s",buf); /* Print the data */
    close(filides[0]);
    exit(EXIT_SUCCESS);
```

Thinking 6.2

`dup` 函数的功能是将一个文件描述符（例如`fd0`）所对应的内容映射到另一个文件描述符（例如`fd1`）中。这个函数最终会将`fd0`和`pipe`的引用次数都增加1，将`fd1`的引用次数变为`fd0`的引用次数。若在复制了文件描述符页面后产生了**时钟中断**，`pipe`的引用次数没来的及增加，可能会导致另一进程调用 `pipeisclosed`，发现 `pageref(fd[0]) = pageref(pipe)`，误以为读/写端已经关闭。

Thinking 6.3

在进行系统调用时，系统陷入内核，会关闭时钟中断。

```
.macro CLI
    mfc0 t0, CP0_STATUS
    li t1, (STATUS_CU0 | 0x1)
    or t0, t1
    xor t0, 0x1
    mtc0 t0, CP0_STATUS
.endm
```

Thinking 6.4

可以解决，若在 `pageref(pipe) > pageref(fd)` 的情况下便没有问题，而如果 `pageref(pipe) == pageref(fd)` 的话，那么当读缓冲区为空，写缓冲区为满时会**再次循环**直到进程切换两者全部unmap为止。

`dup` 也会出现同样的问题，先对 `pipe` 进行map，再对 `fd` 进行map即可。

Thinking 6.5

当加载到 `bin_size~sgsize` 之间的数据时，就知道新入了 `bss` 端，使用 `bzero` 函数赋值为0，不需要再读取ELF的数据。

Thinking 6.6

在`user/user.lids`文件中约定了text段地址为0x00400000

Thinking 6.7

我们用到的shell命令是外部命令，因为我们的user文件夹中有`cat.c` `ls.c` 文件，Linux下的cd指令没有对应的文件，使用时也不需要单独的创建一个子进程。cd所做的是改变 shell 的 **PWD**。因此倘若 cd 是一个外部命令，那么它改变的将会是子 shell 的 PWD，也不会向父 shell 返回任何东西。所以，当前 shell 的 PWD 就不会做任何改变。**所有能对当前 shell 的环境作出改变的命令都必须是内部命令**。因此如果我们将 cd 做成外部命令，就无法像原来一样改变当前目录了。

Thinking 6.8

```
if ((r = dup(0, 1)) < 0)
    user_panic("dup: %d", r);
```

Thinking 6.9

两次，分别对应 `[00001c03] SPAWN: ls.b`、`[00002404] SPAWN: cat.b`

四次，分别对应 `[00003406] destroying 00003406`、`[00002c05] destroying 00002c05`、`[00002404] destroying 00002404`、`[00001c03] destroying 00001c03`